

Mamba: 基于选择性状态空间的线性时间序列建模

论文精读与全文解读

基于 Albert Gu & Tri Dao 原论文 (arXiv:2312.00752v2)

March 6, 2026

Contents

1 摘要 (Abstract)	3
2 引言 (Introduction)	3
2.1 选择机制 (Selection Mechanism)	4
2.2 硬件感知算法 (Hardware-aware Algorithm)	4
2.3 架构 (Architecture)	4
2.4 Mamba的关键特性	5
2.5 实验验证概述	5
3 状态空间模型 (State Space Models)	6
3.1 SSM的三种计算形式	6
3.2 离散化 (Discretization)	8
3.3 计算方式 (Computation)	8
3.4 线性时间不变性 (Linear Time Invariance, LTI)	8
3.5 结构与维度 (Structure and Dimensions)	9
3.6 通用状态空间模型	9
3.7 SSM 架构 (SSM Architectures)	9
4 选择性状态空间模型 (Selective State Space Models)	10
4.1 动机:	10
4.2 用选择性改进 SSM	11
4.3 选择性 SSM 的高效实现	12
4.4 简化的 SSM 架构	13
4.5 选择机制的性质	14
5 实证评估 (Empirical Evaluation)	15
5.1 合成任务 (Synthetic Tasks)	16
5.1.1 选择性复制	16
5.1.2 归纳头	16
5.2 语言建模 (Language Modeling)	17
5.2.1 缩放定律 (Scaling Laws)	17
5.2.2 下游评估 (Downstream Evaluations)	18
5.3 DNA 建模 (DNA Modeling)	19
5.4 音频建模与生成 (Audio Modeling and Generation)	20
5.5 速度和内存基准 (Speed and Memory Benchmarks)	21

5.6 模型消融 (Model Ablations)	22
5.6.1 架构消融	22
5.6.2 选择性参数消融	23
6 讨论 (Discussion)	24
7 结论 (Conclusion)	24
A 选择性 SSM 的机制 (Mechanics of Selective SSMs)	25
B 选择性 SSM 的硬件感知算法	25
C 讨论:	25
D 扩展的相关工作 (Extended Related Work)	26
E 实验细节 (Experimental Details)	26
E.1 合成任务细节	26
E.2 语言建模细节	26
E.3 附加缩放定律消融	27
E.4 DNA 建模细节	27
E.5 音频细节	27
E.6 效率基准细节	28
F 参考文献一览表	29

1 摘要 (Abstract)

原文翻译

基础模型 (Foundation Models) , Transformer 架构及其核心的注意力 (Attention) 许多亚二次时间复杂度的架构——如线性注意力 门控卷积和循环模型 以及结构化状态空间模型 (SSMs) ——已被开发出来以解决 Transformer 在长序列上的计算低效问题, 我们发现, **基于内容的推理** (content-based reasoning) , 首先, SSM 参数成为输入的函数, , token **选择性地**传播或遗忘沿序列长度维度的信息 其次, , **硬件感知并行算法**。 我们将这些选择性 SSM 整合到一个简化的端到端神经网络架构中, MLP 块 (**Mamba**) Mamba 享有快速推理 (Transformer 高 $5\times$ 的吞吐量) , 作为通用序列模型骨架, Mamba 在语言 音频和基因组学等多种模态上达到了最先进的性能 在语言建模上, Mamba-3B 模型超越了同等规模的 Transformer, Transformer 的性能,

通俗解释

这篇论文提出了一种叫做 **Mamba** 的新型序列建模架构, **选择性状态空间模型 (Selective SSM)** 。 用一个比喻来说: Transformer 就像一个有无限记忆力但查找缓慢的图书管理员——它记住了所有书的位置 () , ($O(n^2)$ 复杂度) 而之前的 SSM 就像一个记忆力有限但查找极快的管理员——它只能记住固定的几个信息 () , ($O(n)$ 复杂度) Mamba 的创新在于: ”快速管理员”学会**选择性记忆**——根据当前看到的内容, 哪些可以忘掉 这样既保持了线性时间的速度优势, Transformer 的建模能力 结果: 3B 参数的 Mamba 模型能匹配 6-7B 参数 Transformer 的表现, $5\times$

关键概念

- **核心问题:** , Transformer 级别的建模能力?
- **核心方案:** ——让 SSM 参数随输入动态变化 ()
- **技术挑战:** (LTI) , ;
- **架构简化:** H3 架构的 SSM 块与 Transformer 的 MLP 块合并为单一的 Mamba 块
- **关键结果:** Mamba-3B $\approx$ Transformer-7B 的性能, $5\times$ 。

2 引言 (Introduction)

原文翻译

基础模型 (FM) , , 这些 FM 的骨架通常是**序列模型**, 图像 语音 音频 时间序列和基因组学等各种领域的任意输入序列上运行 Sutskever et al. 2014 是 seq2seq 的开创工作; Dosovitskiy et al. 2020 是 ViT; van den Oord et al. 2016 是 WaveNet; Brown et al. 2020 是 GPT-3; Ismail Fawaz et al. 2019 是时间序列深度学习综述; Poli et al. 2023 是 Hyena 模型 虽然这一概念与特定的模型架构选择无关, FM 主要基于一种序列模型: Transformer 及其核心注意力层 Vaswani et al. 2017 提出

了 Transformer 架构；Bahdanau et al. 2015 提出了注意力机制 自注意力的有效性归因于其在上下文窗口内密集路由信息的能力，然而，：，大量研究致力于开发更高效的注意力变体来克服这些缺陷，迄今为止，

最近，**结构化状态空间序列模型 (SSMs)** 已成为序列建模的一类有前途的架构 Gu et al. 2021 提出 LSSL/S4 的前身；Gu et al. 2022 提出 S4 模型 这些模型可以被解释为循环神经网络 (RNN) (CNN) ， Kalman 1960 是经典卡尔曼滤波 这类模型可以非常高效地计算为循环或卷积形式，此外， (Long Range Arena) 许多 SSM 变体在音频和视觉等连续信号数据领域取得了成功 然而， () 我们提出一类新的**选择性状态空间模型**，， Transformer 的建模能力同时在序列长度上线性缩放

通俗解释

引言首先铺设了背景： Transformer。Transformer 强大但有两个致命缺陷——只能看有限窗口 计算量随窗口大小平方增长 虽然很多人尝试改进注意力效率，然后介绍了 SSM () ——一种结合了 RNN 和 CNN 优点的新型架构 SSM 在连续数据 (视觉) ， () 本文的贡献就是提出”选择性 SSM”来解决这个问题

2.1 选择机制 (Selection Mechanism)

原文翻译

首先，： **选择数据的能力** () 基于选择性复制和归纳头等重要合成任务的直觉，——通过将 SSM 参数基于输入进行参数化 这使得模型能够过滤掉不相关的信息并无限期地记住相关信息

2.2 硬件感知算法 (Hardware-aware Algorithm)

原文翻译

这个简单的改变给模型的计算带来了技术挑战；， SSM 模型必须是时间不变和输入不变的才能保证计算效率 我们通过一种硬件感知算法克服了这一点——该算法使用扫描 (scan) ， GPU 内存层次结构的不同级别之间显式存储扩展的状态 由此产生的实现在理论上 (， SSM 是伪线性的) (A100 GPU 上快达 3×)

2.3 架构 (Architecture)

原文翻译

我们简化了先前的深度序列模型架构， SSM 架构的设计与 Transformer 的 MLP 块组合为单一块， (Mamba) ，

通俗解释

引言的三个核心贡献对应三个段落：

1. **选择机制**：SSM 的参数 (Δ 、 B 、 C) ， “看内容办事”——遇到重要 token 就记住，
2. **硬件感知算法**：LTI 性质，FFT 卷积 解决方案是用并行扫描 (parallel scan) ， GPU 内存层级优化 (HBM $\leftrightarrow$ SRAM 的数据搬运)
3. **架构简化**：SSM 块和 MLP 块， Mamba 块，

2.4 Mamba的关键特性

原文翻译

选择性 SSM， Mamba 架构， ， ： (i) **高质量**：
(ii) **快速训练和推理**： ， ， (iii) **长上下文**：
1M 的性能提升

2.5 实验验证概述

原文翻译

我们在预训练质量和特定领域任务性能方面， Mamba 作为通用序列 FM 骨架的潜力：

- **合成任务**： ， Mamba 不仅轻松解决了它们， **将解决方案无限外推** (>1M tokens)
- **音频和基因组学**：Mamba 在音频波形和 DNA 序列建模上超越了 SaShiMi、Hyena 和 Transformer 等先前最先进模型， 在两种设置中， **性能随着更长上下文持续提升直到百万级序列长度**。
- **语言建模**：Mamba 是第一个真正达到 **Transformer 级别性能的线性时间序列模型**。通过高达 1B 参数的缩放定律， Mamba 超越了包括基于 LLaMa 的非常强大的现代 Transformer 训练配方在内的大范围基线 我们的 Mamba 语言模型比同等规模的 Transformer 有 5 $\times$ 的生成吞吐量， Mamba-3B 的质量匹配两倍规模 Transformer (， Pythia-3B 高 4 个百分点， Pythia-7B)

模型代码和预训练检查点已在 <https://github.com/state-spaces/mamba> 开源

关键概念

引言核心要点总结：

- Transformer 的注意力机制强大但效率低 ($O(L^2)$)
- SSM 高效 ($O(L)$) ，
- 选择性 SSM 通过让参数依赖输入来解决这个问题
- 硬件感知的并行扫描算法解决了由此带来的计算挑战
- Mamba 架构简洁 高效， /音频/基因组学上全面达到或超越 SOTA。

3 状态空间模型 (State Space Models)

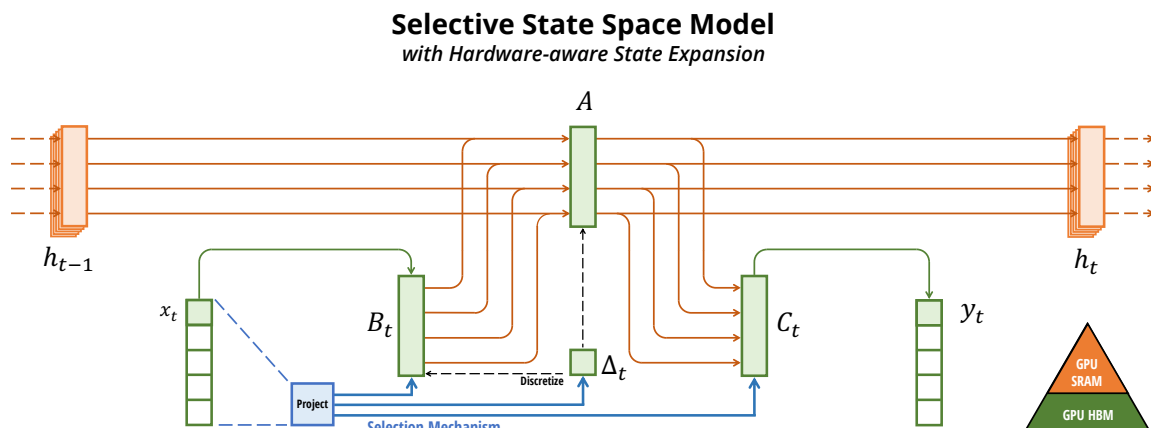


Figure 1: (概览) 结构化 SSM 独立地将输入 x 的每个通道 ($D = 5$) 先前 SSM 通过巧妙的替代计算路径 ((Δ, A, B, C) 参数在所有时间步恒定) (DN 乘以批大小 B 和序列长度 L) 我们的选择机制重新引入了输入依赖的动态, GPU 内存层次的更高效级别中显式存储扩展状态

原文翻译

结构化状态空间序列模型 (S4) , RNN、CNN 和经典状态空间模型 它们的灵感来自一个特定的连续系统, $h(t) \in \mathbb{R}^N$ 将一维函数或序列 $x(t) \in \mathbb{R}$ 映射到 $y(t) \in \mathbb{R}$ 。具体而言, S4 模型由四个参数 (Δ, A, B, C) 定义,

通俗解释

图 1 展示了 SSM 的核心思想:

- 输入 x : $D = 5$ 个通道,
- 隐状态 h : $N = 4$ 维隐状态, $D \times N = 20$ 。
- 输出 y : ,
- 先前 SSM 的参数是固定的 () ,
- 选择性 SSM 的参数随输入变化,

3.1 SSM的三种计算形式

原文翻译

S4 模型定义了三种等价的计算形式:
连续形式 (ODE) :

$$h'(t) = Ah(t) + Bx(t) \quad (1)$$

$$y(t) = Ch(t) \quad (2)$$

离散化递归形式：

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t \quad (3)$$

$$y_t = Ch_t \quad (4)$$

全局卷积形式：

$$\bar{K} = (C\bar{B}, C\bar{A}\bar{B}, \dots, C\bar{A}^k\bar{B}, \dots) \quad (5)$$

$$y = x * \bar{K} \quad (6)$$

数学推导详解

SSM 的三种形式详细推导：

1. 连续形式是一个常微分方程 (ODE)：

- $A \in \mathbb{R}^{N \times N}$ ： ,
- $B \in \mathbb{R}^{N \times 1}$ ： ,
- $C \in \mathbb{R}^{1 \times N}$ ： ,
- $h(t) \in \mathbb{R}^N$ ：

2. 离散化将连续 ODE 变为离散递推 使用零阶保持 (ZOH)：

$$\bar{A} = \exp(\Delta A) \quad (7)$$

$$\bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I) \cdot \Delta B \quad (8)$$

其中 Δ 是步长参数 当 A 是对角矩阵时,

数值例子： $N = 1$, $A = -1$, $B = 1$, $\Delta = 0.1$,：

$$\bar{A} = e^{0.1 \times (-1)} = e^{-0.1} \approx 0.905, \quad \bar{B} \approx \frac{e^{-0.1} - 1}{-0.1} \times 0.1 \times 1 \approx 0.095.$$

递推变为 $h_t = 0.905 h_{t-1} + 0.095 x_t$, (EMA)

3. 卷积形式的推导：

$$\begin{aligned} h_0 &= \bar{B}x_0, & y_0 &= C\bar{B}x_0 \\ h_1 &= \bar{A}\bar{B}x_0 + \bar{B}x_1, & y_1 &= C\bar{A}\bar{B}x_0 + C\bar{B}x_1 \\ h_2 &= \bar{A}^2\bar{B}x_0 + \bar{A}\bar{B}x_1 + \bar{B}x_2, & y_2 &= C\bar{A}^2\bar{B}x_0 + C\bar{A}\bar{B}x_1 + C\bar{B}x_2 \end{aligned}$$

可以看出 $y = x * \bar{K}$, $\bar{K}_k = C\bar{A}^k\bar{B}$ 。

注意事项

关键区别： (Δ, A, B, C) 不随时间变化 (LTI 条件) 一旦参数随输入改变 (SSM), ,

3.2 离散化 (Discretization)

原文翻译

第一阶段将“连续参数” $(\Delta, \mathbf{A}, \mathbf{B})$ 通过固定公式转换为“离散参数” $(\overline{\mathbf{A}}, \overline{\mathbf{B}})$, $\overline{\mathbf{A}} = f_A(\Delta, \mathbf{A})$, $\overline{\mathbf{B}} = f_B(\Delta, \mathbf{A}, \mathbf{B})$, (f_A, f_B) 称为离散化规则
可以使用各种规则, (ZOH):

$$\overline{\mathbf{A}} = \exp(\Delta \mathbf{A}), \quad \overline{\mathbf{B}} = (\Delta \mathbf{A})^{-1}(\exp(\Delta \mathbf{A}) - \mathbf{I}) \cdot \Delta \mathbf{B} \quad (9)$$

离散化与连续时间系统有深层联系, , 它还与 RNN 的门控机制有联系, 4.5 节重新审视 然而, , SSM 前向传播计算图中的第一步 其他 SSM 变体可以绕过离散化步骤, $(\overline{\mathbf{A}}, \overline{\mathbf{B}})$ 。

通俗解释

离散化就是把连续的微分方程变成离散的递推公式,
类比: “水龙头匀速放水”, “每隔 Δ 时间采样一次水量”。 Δ 越大, ; Δ 越小,
重要的是, Δ 不仅仅是一个技术参数——它和 RNN 中的“门控”概念紧密相关: Δ 大意味着“关注当前输入”(), Δ 小意味着“保持历史状态”() 这正是选择机制的关键

3.3 计算方式 (Computation)

原文翻译

参数从 $(\Delta, \mathbf{A}, \mathbf{B}, \mathbf{C})$ 转换为 $(\overline{\mathbf{A}}, \overline{\mathbf{B}}, \mathbf{C})$ 后, ——线性递推 (3)–(4) 或全局卷积 (5)–(6)。
通常, (), ()

3.4 线性时间不变性 (Linear Time Invariance, LTI)

原文翻译

以上方程的一个重要性质是模型的动态在时间上是恒定的 换句话说, $(\Delta, \mathbf{A}, \mathbf{B}, \mathbf{C})$ 以及因此的 $(\overline{\mathbf{A}}, \overline{\mathbf{B}})$ 对所有时间步都是固定的 这个性质称为**线性时间不变性 (LTI)**,
非正式地, LTI SSM 等价于任何线性递推或卷积, LTI 作为这些模型类的总称
迄今为止, SSM 都是 LTI 的 (), 然而, LTI 模型在建模某些类型数据时存在根本性局限, LTI 约束

关键概念

LTI 的利与弊:

- **优势:** LTI 条件下可以用 FFT 卷积高效并行计算, $O(L \log L)$ 。
- **劣势:** , “选择”——对所有 token 一视同仁
- **Mamba 的突破:** LTI 条件, ,

3.5 结构与维度 (Structure and Dimensions)

原文翻译

结构化 SSM 之所以如此命名, A 矩阵施加结构 最流行的结构形式是**对角矩阵**,
在这种情况下, $A \in \mathbb{R}^{N \times N}$, $B \in \mathbb{R}^{N \times 1}$, $C \in \mathbb{R}^{1 \times N}$ 矩阵都可以用 N 个数表示 为了对批大小 B 、长度 L 、 D 个通道的输入序列 x 进行操作, SSM 独立应用于每个通道 注意在这种情况下, DN , $O(BLDN)$ 的时间和内存; 4.3 节中讨论的基本效率瓶颈的根源

数学推导详解

维度分析:

假设模型维度 $D = 768$, $N = 16$, $L = 2048$, $B = 32$:

- 输入/输出张量: $x, y \in \mathbb{R}^{B \times L \times D}$, $32 \times 2048 \times 768$
- 隐状态张量: $h \in \mathbb{R}^{B \times L \times D \times N}$, $32 \times 2048 \times 768 \times 16$
- 隐状态比输入大 $N = 16$ 倍!
- 总 FLOPs $\approx BLDN = 32 \times 2048 \times 768 \times 16 \approx 8 \times 10^8$

3.6 通用状态空间模型

原文翻译

我们注意到“状态空间模型”这个术语有非常广泛的含义, 它在不同学科中被用来指代许多不同的概念, (MDP,) 动态因果建模 (DCM,) 卡尔曼滤波 () 隐马尔科夫模型 (HMM) (LDS,) 以及广义上的循环 () ()
在本文中, ”SSM”专指结构化 SSM 或 S4 模型这一类别,

3.7 SSM 架构 (SSM Architectures)

原文翻译

SSM 是独立的序列变换, 我们讨论一些最知名的 SSM 架构, :

- **线性注意力**是自注意力的一种近似, SSM 的递推 Katharopoulos et al. 2020 提出线性注意力, softmax 注意力近似为核方法,
- **H3** 将这种递推推广到使用 S4; SSM 的架构 (3) H3 还在主 SSM 层之前插入一个标准的局部卷积 Dao et al. 2023 提出 H3 架构, SSM 架构
- **Hyena** 使用与 H3 相同的架构但用 MLP 参数化的全局卷积替代 S4 层 Poli et al. 2023 提出 Hyena 模型
- **RetNet** 在架构中增加了额外的门控, SSM, Sun et al. 2023 提出 RetNet。】
- **RWKV** 是为语言建模设计的近期 RNN, Attention-Free Transformer。其主要的”WKV”机制涉及 LTI 递推, SSM 的比值 Peng et al. 2023 提出 RWKV。】

其他密切相关的 SSM 和架构在扩展的相关工作 (D) 我们特别强调 S5、QRNN 和 SRU, SSM 最密切相关的方法

4 选择性状态空间模型 (Selective State Space Models)

原文翻译

我们使用合成任务的直觉来激发选择机制 (4.1 节), (4.2 节) 由此产生的时变 SSM 不能使用卷积, 我们通过一种利用现代硬件内存层次结构的硬件感知算法来克服这一点 (4.3 节) 然后我们描述一种没有注意力甚至 MLP 块的简单 SSM 架构 (4.4 节) 最后, (4.5 节)

4.1 动机:

原文翻译

我们论证序列建模的一个根本问题是**将上下文压缩到更小的状态中**。事实上, 例如, , 不压缩上下文 这可以从自回归推理需要显式存储整个上下文 (KV 缓存) , Transformer 的慢线性时间推理和二次方时间训练 另一方面, , 然而, 为了理解这个原理, (2)

- **选择性复制 (Selective Copying)** 任务修改了流行的复制任务, token 的位置 它需要**内容感知**的推理来记忆相关 token (彩色) token (白色)
- **归纳头 (Induction Heads)** 任务是一种众所周知的机制, LLM 大部分上下文学习能力的原因 它需要**上下文感知**的推理来知道何时在适当的上下文 (黑色)

这些任务揭示了 LTI 模型的失败模式 从递推的角度看, ($(\overline{A}, \overline{B})$ 转移) , 从卷积的角度看, (), () 更具体地说, , 总之, : , 据此, **选择性:**

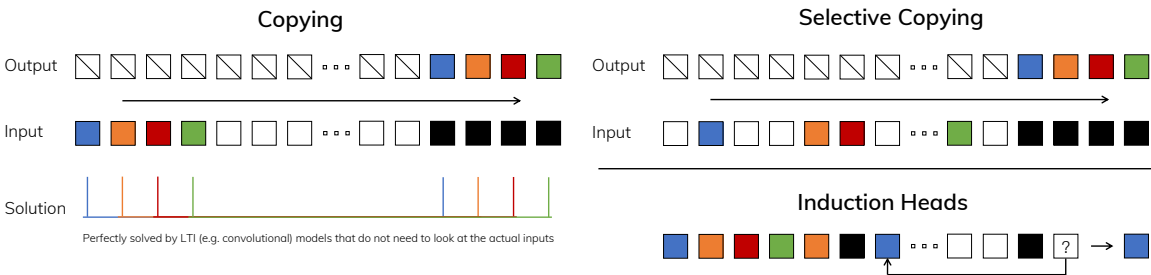


Figure 2: (左) , () (右上) , (右下)

通俗解释

图 2 展示了两个关键的合成任务:
选择性复制任务 (): , (), () 你的任务是按顺序记住彩色

球的颜色 关键难点在于，

- LTI 模型 () ——相当于只会“数拍子”。
- 选择性模型可以根据内容决定记不记——看到彩色球就记，

归纳头任务 () : “A B”这个组合， “A”时应该预测“B”。这需要联想记忆——模型不仅要记住内容，
这两个任务解释了为什么 LTI () , ()

4.2 用选择性改进 SSM

原文翻译

将选择机制融入模型的一种方法是让影响序列交互的参数 (RNN 的递推动态或 CNN 的卷积核)

算法 1 和 2 展示了我们使用的主要选择机制 主要区别简单地是让几个参数 Δ 、 B 、 C 成为输入的函数， 特别地， L ， 这失去了与卷积的等价性，

我们具体选择 $s_B(x) = \text{Linear}_N(x)$, $s_C(x) = \text{Linear}_N(x)$, $s_\Delta(x) = \text{Broadcast}_D(\text{Linear}_1(x))$, $\tau_\Delta = \text{softplus}$, Linear_d 是到维度 d 的参数化投影 s_Δ 和 τ_Δ 的选择源于与 RNN 门控机制的联系

Algorithm 1 SSM (S4)

Require: $x : (B, L, D)$

Ensure: $y : (B, L, D)$

- 1: $A : (D, N) \leftarrow \text{Parameter}$ ▷ 表示结构化 $N \times N$ 矩阵
- 2: $B : (D, N) \leftarrow \text{Parameter}$
- 3: $C : (D, N) \leftarrow \text{Parameter}$
- 4: $\Delta : (D) \leftarrow \tau_\Delta(\text{Parameter})$
- 5: $\overline{A}, \overline{B} : (D, N) \leftarrow \text{discretize}(\Delta, A, B)$
- 6: $y \leftarrow \text{SSM}(\overline{A}, \overline{B}, C)(x)$ ▷ 时间不变:
- 7: **return** y

Algorithm 2 SSM + Selection (S6)

Require: $x : (B, L, D)$

Ensure: $y : (B, L, D)$

- 1: $A : (D, N) \leftarrow \text{Parameter}$ ▷ 表示结构化 $N \times N$ 矩阵
- 2: $B : (B, L, N) \leftarrow s_B(x)$
- 3: $C : (B, L, N) \leftarrow s_C(x)$
- 4: $\Delta : (B, L, D) \leftarrow \tau_\Delta(\text{Parameter} + s_\Delta(x))$
- 5: $\overline{A}, \overline{B} : (B, L, D, N) \leftarrow \text{discretize}(\Delta, A, B)$
- 6: $y \leftarrow \text{SSM}(\overline{A}, \overline{B}, C)(x)$ ▷ 时变: (*scan*)
- 7: **return** y

通俗解释

算法 1 (S4) 2 (S6) :

- 第 1 行 () : A 是固定可学习参数, (D, N) , D 是模型通道数, N 是 SSM 状态

维度

• 第 2 行 (关键不同) :

- S4 中 B 是固定参数, (D, N) ——对所有时间步相同
- S6 中 $B = s_B(x) = \text{Linear}_N(x)$, (B, L, N) ——每个 batch、每个时间步都不同!

• 第 3 行 (关键不同) : C 的变化与 B 类似

• 第 4 行 (关键不同) : Δ 从 (D) 变为 (B, L, D) 。 $s_\Delta(x)$ 先投影到维度 1, D 。

• 第 5 行 (维度膨胀) : (D, N) 变为 (B, L, D, N) ——这就是”状态扩展”的巨大内存开销来源

• 第 6 行: S4 可以用递推或卷积; S6 只能用递推 (scan)

数学推导详解

选择性参数的具体计算:

假设输入 $x \in \mathbb{R}^{B \times L \times D}$, $D = 768$, $N = 16$:

B 的选择性投影: $s_B(x) = x \cdot W_B$, $W_B \in \mathbb{R}^{D \times N} = \mathbb{R}^{768 \times 16}$ 。结果 $B \in \mathbb{R}^{B \times L \times N} = \mathbb{R}^{B \times L \times 16}$ 。注意这里 B 对每个通道 D 是共享的

Δ 的选择性投影:

1. 先投影到维度 R () : $s_\Delta^{(1)}(x) = x \cdot W_\Delta \in \mathbb{R}^{B \times L \times R}$, $R \ll D$ 。
2. 再广播到维度 D : $s_\Delta(x) = \text{Broadcast}_D(s_\Delta^{(1)}(x)) \in \mathbb{R}^{B \times L \times D}$ 。
3. 应用 softplus: $\Delta = \text{softplus}(\text{bias} + s_\Delta(x))$, $\Delta > 0$ 。

softplus 函数: $\text{softplus}(x) = \log(1 + e^x)$, ReLU ,

4.3 选择性 SSM 的高效实现

原文翻译

硬件友好的基元如卷积和注意力已被广泛应用 我们的目标是使选择性 SSM 在现代硬件 (GPU) 选择机制相当自然, Δ 在递推 SSM 中随时间变化 然而, S4 及其所有衍生都使用了 LTI () , 选择机制旨在克服 LTI 模型的局限性; , SSM 的计算问题 我们用三种经典技术来解决: **核融合** (kernel fusion) **并行扫描** (parallel scan) **重计算** (recomputation) 我们做出两个主要观察:

- 朴素递推计算使用 $O(BLDN)$ FLOPs, $O(BLD \log(L))$ FLOPs, 因此对于长序列和不太大的状态维度 N , FLOPs。
- 两个挑战是递推的顺序性和大内存使用 对于后者, h 。

主要思想是利用现代加速器 (GPU) , h 。特别地, () 这包括我们的扫描操作, IO 量, 具体而言, GPU HBM () (B, L, D, N) 的扫描输入 $(\overline{A}, \overline{B})$, : HBM 加载 SSM 参数 (Δ, A, B, C) 到快速 SRAM, SRAM 中执行离散化和递推, (B, L, D) 的最终输出写回 HBM。

为避免顺序递推，
最后，
HBM 加载输入到 SRAM 时重新计算 因此，
实现具有相同的内存需求

通俗解释

这部分是 Mamba 的工程核心。问题是：
 h 的大小是 (B, L, D, N) ， N 倍，
解决方案用了三个技巧：

1. 核融合 (Kernel Fusion) —— 把多个操作合成一个 GPU 核函数：

- 不融合时：HBM 读参数 $\rightarrow$ 写离散化结果到 HBM $\rightarrow$ 再读出来做 scan $\rightarrow$ 写结果到 HBM。大量 IO！
- 融合后：HBM 读参数到 SRAM $\rightarrow$ 在 SRAM 内做离散化 + scan + 乘 C $\rightarrow$ 只写最终结果回 HBM。IO 减少 N 倍！

GPU 内存层次：HBM 容量大 (80GB) ；SRAM 容量小 (~20MB) 类比：HBM 是仓库，SRAM 是工作台

2. 并行扫描 (Parallel Scan) —— 把顺序递推并行化 虽然 h_t 依赖 h_{t-1} ，
 $O(\log L)$ 步完成

3. 重计算 (Recomputation) —— 反向传播时不从 HBM 读中间状态 ()，
SRAM 计算 虽然计算了两次，IO，！

注意事项

为什么重计算比存储更快？ 因为 GPU 上大部分操作是 IO 密集 (memory-bound) 计算密集 (compute-bound) 扫描操作的瓶颈不在 FLOPs 而在数据搬运 重算避免了读写 $O(BLDN)$ 大小的中间张量，IO 时间远超重算的计算时间

4.4 简化的 SSM 架构

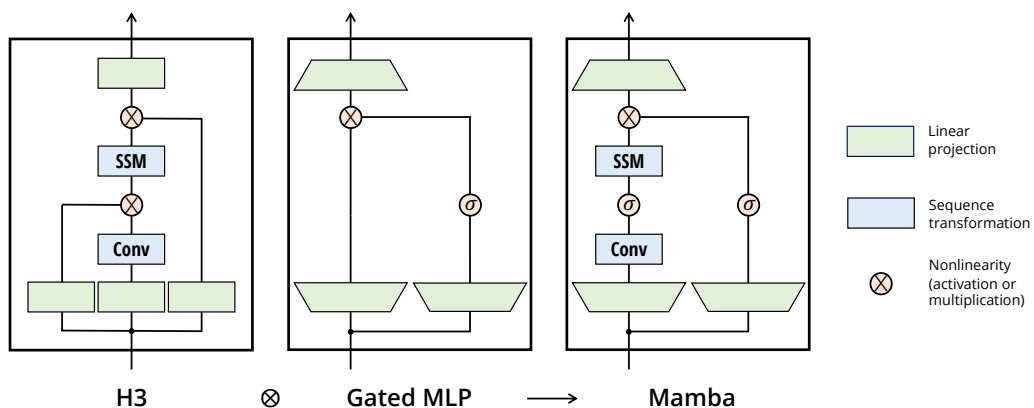


Figure 3: (架构。) H3 块 (SSM 架构的基础) MLP 块组合 不再交替堆叠这两种块，Mamba 块与 H3 块相比，Mamba 用激活函数替代第一个乘法门控 与 MLP 块相比，Mamba 在主分支上添加了 SSM。 σ 使用 SiLU/Swish 激活函数

原文翻译

与结构化 SSM 一样，SSM 是独立的序列变换，H3 架构是最知名 SSM 架构的基础，MLP () 我们通过将这两个组件合并为一个来简化这个架构，(3) 这受到了门控注意力单元 (GAU) ，这个架构涉及将模型维度 D 扩展一个可控的扩展因子 E 。对于每个块， $(3ED^2)$ ($2ED^2$ 用于输入投影， ED^2 用于输出投影)，SSM 贡献较少我们始终固定 $E = 2$ ，Transformer 交替的 MHA 和 MLP 块的 $12D^2$ 参数 我们使用 SiLU/Swish 激活函数，MLP 变成了流行的”SwiGLU”变体 此外，(LayerNorm)

通俗解释

图 3 展示了三种架构的对比：

H3 块 () : ———一路经过 SSM 处理，————最后合并输出 还需要额外的 MLP 块

MLP 块 () : Transformer MLP, SwiGLU: $\text{output} = (\sigma(xW_1) \odot xW_2)W_3$ 。

Mamba 块 () : H3 的 SSM 路径和 MLP 的结构合二为一！

1. 输入 x 经过线性投影扩展到维度 ED ($E = 2$)
2. 主分支: $\text{Conv1d} \rightarrow \text{SiLU} \rightarrow \text{SSM}$ 。
3. 旁路分支: SiLU ()
4. 两路相乘 () , D 。

参数量匹配: Transformer 层 = MHA + MLP $\approx 4D^2 + 8D^2 = 12D^2$ 。两个 Mamba 块 $= 2 \times 6D^2 = 12D^2$ 。所以参数量等价

4.5 选择机制的性质

原文翻译

选择机制是一个更广泛的概念，我们强调最重要的联系：RNN 的经典门控机制是我们 SSM 选择机制的一个特例

定理 1 (门控联系). 当 $N = 1, \mathbf{A} = -1, \mathbf{B} = 1, s_\Delta = \text{Linear}(x), \tau_\Delta = \text{softplus}$ 时, SSM 递推 (2) :

$$g_t = \sigma(\text{Linear}(x_t)) \quad (10)$$

$$h_t = (1 - g_t)h_{t-1} + g_tx_t \quad (11)$$

数学推导详解

定理 1 的证明:

设 $N = 1, \mathbf{A} = -1, \mathbf{B} = 1$ 。连续 SSM 变为漏积分器: $h'(t) = -h(t) + x(t)$ 。

步骤 1: Δ_t :

$$\Delta_t = \text{softplus}(\text{Linear}(x_t)) = \log(1 + e^{\text{Linear}(x_t)})$$

步骤 2: ZOH 离散化 $\overline{\mathbf{A}}$:

$$\begin{aligned}\overline{\mathbf{A}}_t &= \exp(\Delta_t \cdot \mathbf{A}) = \exp(-\Delta_t) = \exp(-\log(1 + e^{\text{Linear}(x_t)})) \\ &= \frac{1}{1 + e^{\text{Linear}(x_t)}} = \sigma(-\text{Linear}(x_t)) = 1 - \sigma(\text{Linear}(x_t))\end{aligned}$$

步骤 3: ZOH 离散化 $\overline{\mathbf{B}}$:

$$\begin{aligned}\overline{\mathbf{B}}_t &= (\Delta \mathbf{A})^{-1}(\exp(\Delta \mathbf{A}) - \mathbf{I}) \cdot \Delta \mathbf{B} \\ &= \frac{1}{-\Delta_t}(\exp(-\Delta_t) - 1) \cdot \Delta_t \cdot 1 = 1 - \exp(-\Delta_t) = 1 - \overline{\mathbf{A}}_t = \sigma(\text{Linear}(x_t))\end{aligned}$$

步骤 4: $h_t = \overline{\mathbf{A}}_t h_{t-1} + \overline{\mathbf{B}}_t x_t$:

$$h_t = (1 - g_t)h_{t-1} + g_t x_t, \quad \text{其中 } g_t = \sigma(\text{Linear}(x_t))$$

数值例子: $\text{Linear}(x_t) = 2.0$, $g_t = \sigma(2.0) = \frac{1}{1+e^{-2}} \approx 0.88$ 。所以 $h_t \approx 0.12 \cdot h_{t-1} + 0.88 \cdot x_t$ ——当前输入贡献 88%, 12%。当 $g_t \rightarrow 1$ 时完全重置状态, $g_t \rightarrow 0$ 时完全忽略输入

关键概念

选择机制的三种效果:

- 可变间距 (Variable Spacing) : token ("嗯"), $g_t \rightarrow 0$ 时忽略当前输入
- 上下文过滤 (Filtering Context) : ,
- 边界重置 (Boundary Resetting) : , $\Delta_t \rightarrow \infty$ ($g_t \rightarrow 1$) ,

备注 1. 为简洁起见, SSM 缩写为 S6 模型, 选择 (Selection) 扫描 (Scan) S_4 模型

5 实证评估 (Empirical Evaluation)

原文翻译

在第 5.1 节, Mamba 解决第 4.1 节中提出的两个合成任务的能力 然后我们在三个领域进行评估, : () ; DNA 序列预训练和长序列分类任务微调; 最后展示 Mamba 在训练和推理时的计算效率, SSM 的各个组件

5.1 合成任务 (Synthetic Tasks)

5.1.1 选择性复制

原文翻译

复制任务是序列建模中研究最充分的合成任务之一，如第 4.1 节所述，LTI SSM 可以通过只跟踪时间而不推理数据来轻松解决此任务；，选择性复制任务通过随机化 token 之间的间距来阻止这种捷径 表 1 确认，（ H3 和 Mamba），（ S4 修改为 S6），

Table 1: （选择性复制。）

Model	Arch.	Layer	Acc.
S4	No gate	S4	18.3
-	No gate	S6	97.0
H3	H3	S4	57.0
Hyena	H3	Hyena	30.1
-	H3	S6	99.7
-	Mamba	S4	56.4
-	Mamba	Hyena	28.4
Mamba	Mamba	S6	99.8

通俗解释

表 1 的关键发现：

- **S6 层是决定性因素：**（ H3、Mamba），S4/Hyena 换成 S6，97%+。
- **架构门控有帮助但不够：**H3 和 Mamba 架构比无门控的 S4 好（57% vs 18%），S6。
- **Hyena 反而更差：**Hyena 的全局卷积在这个任务上甚至不如 S4（30.1% vs 57.0%），MLP 参数化的卷积核更难学到正确的选择模式

结论：选择性是在序列层内部解决的，

5.1.2 归纳头

原文翻译

归纳头是一个来自机制可解释性视角的简单任务，LLM 的上下文学习能力 它要求模型执行联想回忆和复制：，”Harry Potter”这个双词组，”Harry”出现时，”Potter”。

图 4 显示，Mamba——更准确地说，SSM 层——有能力完美解决此任务，token 同时忽略中间的一切 它完美泛化到百万级长度序列，4000×，2×。

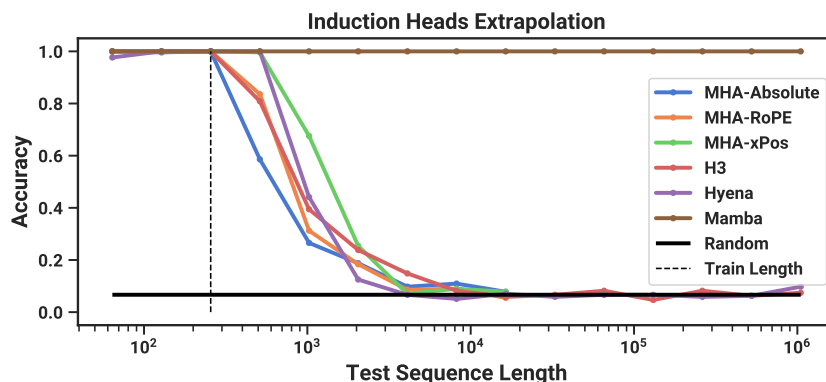


Figure 4: (归纳头。) $2^8 = 256$ 上训练, $2^6 = 64$ 到 $2^{20} = 1048576$ 的递增序列长度上测试

通俗解释

图 4 展示了各模型在归纳头任务上随序列长度增长的准确率:

- 横轴: (), 64 到 1048576。
- 纵轴:
- Mamba (S6): 100% 准确率——从训练长度 256 一直外推到 ~100 万!
- 注意力模型 (MHA-Abs/RoPE/xPos): , $2\times$ 后急剧下降, 2^{14} 。
- H3/Hyena:

这个结果非常惊人: Mamba 不仅解决了归纳头, 无限外推——这是任何注意力模型都做不到的

5.2 语言建模 (Language Modeling)

原文翻译

我们在标准自回归语言建模上评估 Mamba 架构, , () 我们设置模型尺寸以镜像 GPT3 规格 使用 Pile 数据集, GPT-3 所述的训练配方

5.2.1 缩放定律 (Scaling Laws)

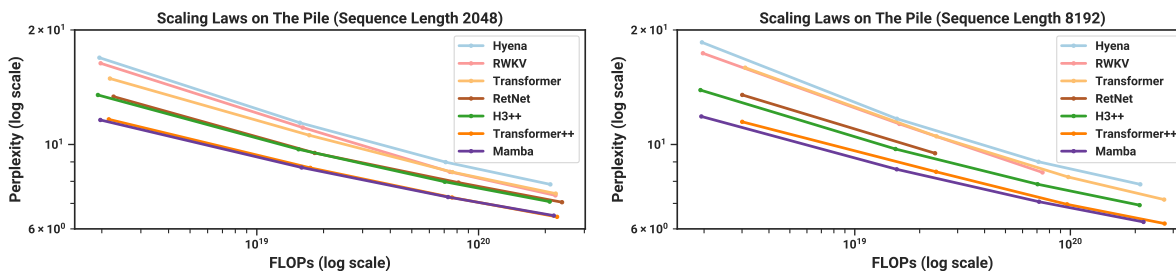


Figure 5: (缩放定律。) $\approx 125M$ 到 $\approx 1.3B$ 参数的模型, Pile 上训练 Mamba 比所有其他无注意力模型缩放更好, "Transformer++"配方性能模型,

原文翻译

对于基线，Transformer 架构（GPT3 架构），Transformer 配方（Transformer++），PaLM 和 LLaMa 架构（SwiGLU MLP、RMSNorm 替代 Layer-Norm、非线性偏置 更高学习率等）我们还比较其他近期亚二次方架构（5）图 5 显示了在标准 Chinchilla 协议下， $\approx 125M$ 到 $\approx 1.3B$ 参数模型的缩放定律 Mamba 是第一个匹配非常强的 Transformer 配方（Transformer++），

通俗解释

图 5 是论文最重要的结果之一：
左图（2K）：
• 横轴是参数量（），Pile 验证集困惑度（）
• Mamba（）Transformer++（），Hyena、H3++、RWKV、RetNet 等
右图（8K）：
• 更长上下文时差距更明显：Mamba 保持甚至超越 Transformer++ 的性能
• RWKV 和 RetNet 因缺乏高效实现而缺少 8K 结果
关键信息：Mamba 不仅仅是“还行的替代品”，匹配了最优化的 Transformer。

5.2.2 下游评估 (Downstream Evaluations)

原文翻译

表 2 展示了 Mamba 在一系列流行的下游零样本评估任务上的性能 我们与这些规模上最知名的开源模型比较，Pythia 和 RWKV，数据集和训练长度（300B tokens）

Table 2: （零样本评估。）对比使用各种分词器 训练最多 300B tokens 的开源 LM。对于每个模型规模，Mamba 在每一项评估结果上都是同类最佳，

Model	Token.	Pile ppl↓	LAMBADA ppl↓	LAMBADA acc↑	HellaSwag↑	PIQA↑	Arc-E↑	Arc-C↑	WinoGrande↑	Avg↑
Hybrid H3-130M	GPT2	—	89.48	25.77	31.7	64.2	44.4	24.2	50.6	40.1
Pythia-160M	NeoX	29.64	38.10	33.0	30.2	61.4	43.2	24.1	51.9	40.6
Mamba-130M	NeoX	10.56	16.07	44.3	35.3	64.5	48.0	24.3	51.9	44.7
Hybrid H3-360M	GPT2	—	12.58	48.0	41.5	68.1	51.4	24.7	54.1	48.0
Pythia-410M	NeoX	9.95	10.84	51.4	40.6	66.9	52.1	24.6	53.8	48.2
Mamba-370M	NeoX	8.28	8.14	55.6	46.5	69.5	55.1	28.0	55.3	50.0
Pythia-1B	NeoX	7.82	7.92	56.1	47.2	70.7	57.0	27.1	53.5	51.9
Mamba-790M	NeoX	7.33	6.02	62.7	55.1	72.1	61.2	29.5	56.1	57.1
GPT-Neo 1.3B	GPT2	—	7.50	57.2	48.9	71.1	56.2	25.9	54.9	52.4
OPT-1.3B	OPT	—	6.64	58.0	53.7	72.4	56.7	29.6	59.5	55.0
Pythia-1.4B	NeoX	7.51	6.08	61.7	52.1	71.0	60.5	28.5	57.2	55.2
RWKV-1.5B	NeoX	7.70	7.04	56.4	52.5	72.4	60.5	29.4	54.6	54.3
Mamba-1.4B	NeoX	6.80	5.04	64.9	59.1	74.2	65.5	32.8	61.5	59.7
GPT-Neo 2.7B	GPT2	—	5.63	62.2	55.8	72.1	61.1	30.2	57.6	56.5
OPT-2.7B	OPT	—	5.12	63.6	60.6	74.8	60.8	31.3	61.0	58.7
Pythia-2.8B	NeoX	6.73	5.04	64.7	59.3	74.0	64.1	32.9	59.7	59.1
RWKV-3B	NeoX	7.00	5.24	63.9	59.6	73.7	67.8	33.1	59.6	59.6
Mamba-2.8B	NeoX	6.22	4.23	69.2	66.1	75.2	69.7	36.3	63.5	63.3
GPT-J-6B	GPT2	—	4.10	68.3	66.3	75.4	67.0	36.6	64.1	63.0
OPT-6.7B	OPT	—	4.25	67.7	67.2	76.3	65.6	34.9	65.5	62.9
Pythia-6.9B	NeoX	6.51	4.45	67.1	64.0	75.2	67.3	35.5	61.3	61.7
RWKV-7.4B	NeoX	6.31	4.38	67.2	65.5	76.1	67.8	37.5	61.0	62.5

通俗解释

表 2 是语言建模的核心实验结果：

关键发现：

- **同规模最佳：** (130M、370M、790M、1.4B、2.8B)，Mamba 在**所有**评估指标上都是最好的
- **匹配两倍规模：** Mamba-1.4B 的平均准确率 59.7% 接近 OPT-2.7B 的 58.7% 和 Pythia-2.8B 的 59.1%——几乎是两倍参数量 Mamba-2.8B (63.3%) GPT-J-6B (63.0%)
- **对比 RWKV：** Mamba-1.4B (59.7%) RWKV-1.5B (54.3%)，RWKV 的线性注意力近似更有效

5.3 DNA 建模 (DNA Modeling)

原文翻译

受大型语言模型成功的启发，DNA 类似于语言——由有限词汇表的离散 token 组成的序列，我们在与近期 DNA 长序列模型相同的设置下，Mamba 作为 FM 骨架进行预训练和微调

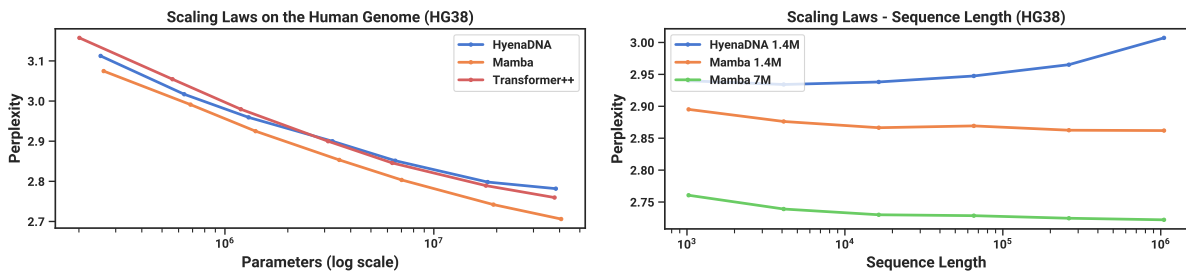


Figure 6: (DNA 缩放定律。) HG38 () 增大规模，Mamba 比基线缩放更好 (右) ， (左) $2^{10} = 1024$ ， $\approx 200K$ 到 $\approx 40M$ 参数与基线不同，Mamba 的选择机制促进了随着上下文长度增加而提升的性能

通俗解释

图 6 展示了 DNA 建模的两个缩放维度：

左图 ()：

- 横轴： 纵轴：
- Mamba 在所有规模上都优于 HyenaDNA 和 Transformer++。
- 在 $\sim 40M$ 参数时，Mamba 达到的困惑度， $3\times-4\times$ 的参数才能匹配

右图 ()：

- 横轴： (1K 到 1M) 纵轴：
- **Mamba 越长越好：** ~ 100 万！
- **HyenaDNA 越长越差：** LTI 全局卷积无法选择性忽略噪声，这完美验证了选择机制的价值——能过滤无关信息的模型，

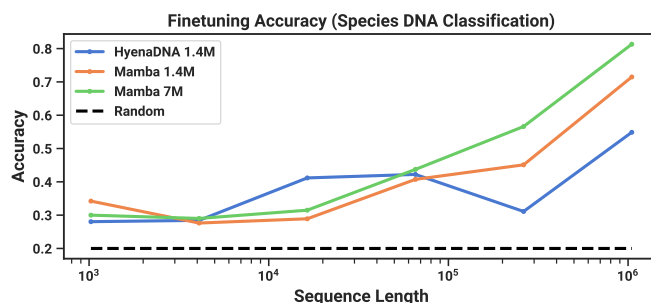


Figure 7: (类人猿 DNA 分类。) , $2^{10} = 1024$ 到 $2^{20} = 1048576$ 的序列上微调后的准确率

通俗解释

图 7 展示了一个极具挑战性的下游任务——区分五种类人猿（黑猩猩 大猩猩 红猩猩 倭黑猩猩）DNA，99% 的基因组！

- 在短序列 ($\leq 2^{14}$) , ——因为信息不足以区分
- 在长序列时差距巨大: Mamba-7M 在 2^{20} 长度达到 **81.31%**, HyenaDNA 只有 54.87%。
- 这说明 Mamba 能有效利用超长上下文中的微小差异来做出判断

5.4 音频建模与生成 (Audio Modeling and Generation)

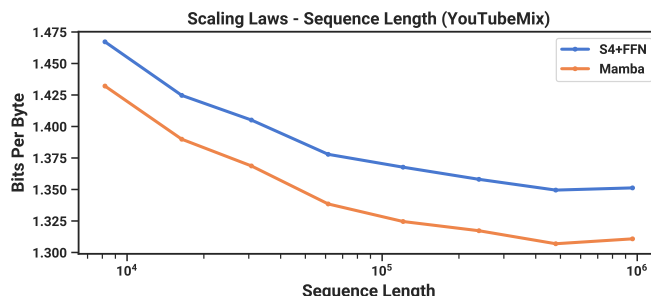


Figure 8: (音频预训练。) Mamba 在自回归音频建模上改善了先前最先进方法 (SaShiMi) , ()

原文翻译

对于音频波形模态, SaShiMi 架构和训练协议比较 该架构是一个 U-Net 骨架, (p), D 翻倍, S4 和 MLP 块 我们考虑用 Mamba 块替换 S4+MLP 块 图 8 评估了训练序列长度从 $2^{13} = 8192$ 增加到 $2^{20} \approx 10^6$ 的效果, Mamba 和 SaShiMi (S4+MLP) 基线都随更长上下文持续改善; Mamba 始终更好,

通俗解释

表 3 展示了语音生成质量的对比:

- **FID** (Fréchet Inception Distance,) : Mamba-6.1M 的 FID 仅 0.94,

Table 3: (SC09)

Model	Params	NLL↓	FID↓	IS↑	mIS↑	AM↓
SampleRNN	35.0M	2.042	8.96	1.71	3.02	1.76
WaveNet	4.2M	1.925	5.08	2.27	5.80	1.47
SaShiMi	5.8M	1.873	1.99	5.13	42.57	0.74
WaveGAN	19.1M	-	2.03	4.90	36.10	0.80
DiffWave	24.1M	-	1.92	5.26	51.21	0.68
+ SaShiMi	23.0M	-	1.42	5.94	69.17	0.59
Mamba	6.1M	1.852	<u>0.94</u>	<u>6.26</u>	<u>88.54</u>	<u>0.52</u>
Mamba	24.3M	<u>1.860</u>	0.67	7.33	144.9	0.36

24M 参数的 DiffWave+SaShiMi (1.42)

- **IS** (Inception Score,) **mIS** (,) : Mamba-24.3M 的 mIS=144.9 几乎是 SaShiMi 最佳结果 (69.17) 2 倍
- 一个小的 6.1M Mamba 模型就超越了所有基线 (19-24M 的 GAN 和扩散模型) , SSM 在音频领域的巨大优势

Table 4: (SC09 模型消融) 6M 参数模型在 SaShiMi 的 U-Net 骨架中,

Outer	Center	NLL↓	FID↓	IS↑	mIS↑	AM↓
S4+MLP	MHA+MLP	1.859	1.45	5.06	47.03	0.70
S4+MLP	S4+MLP	1.867	1.43	5.42	53.54	0.65
S4+MLP	Mamba	1.859	1.42	5.71	56.51	0.64
Mamba	MHA+MLP	1.850	1.37	5.63	58.23	0.62
Mamba	S4+MLP	1.853	<u>1.07</u>	<u>6.05</u>	<u>73.34</u>	<u>0.55</u>
Mamba	Mamba	<u>1.852</u>	0.94	6.26	88.54	0.52

5.5 速度和内存基准 (Speed and Memory Benchmarks)

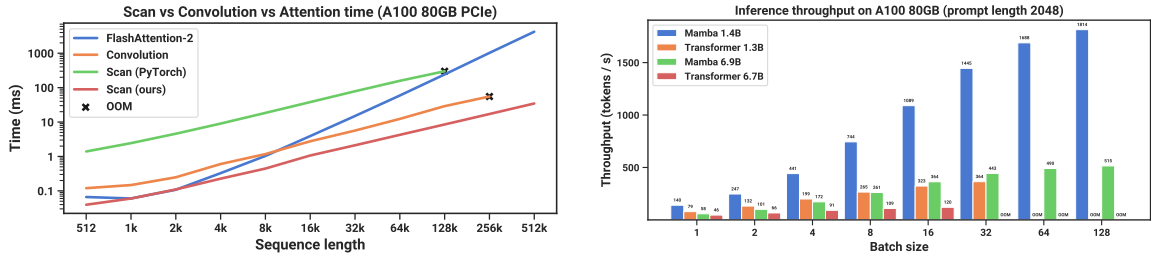


Figure 9: (效率基准。) (左) : scan 比标准实现快 40x。 (右) : , Mamba 可以达到比 Transformer 高 5x 的吞吐量

原文翻译

我们对 SSM scan 操作的速度 ($N = 16$) Mamba 的端到端推理吞吐量进行了基准测试 (9) 我们的高效 SSM scan 在序列长度超过 2K 时比我们所知最快的注意力实现 (FlashAttention-2) , PyTorch 中的标准 scan 实现快 20-40x。Mamba 实现了比同等规模 Transformer 高 4-5x 的推理吞吐量, KV 缓存它可以使用更大的批大小 例如, Mamba-6.9B () 5x 更小的 Transformer-1.3B 更高的推理吞吐量

通俗解释

图 9 展示了 Mamba 的两大效率优势：
左图（ ）：

- 比较了融合 scan、标准 scan、FFT 卷积和 FlashAttention-2。
- 融合 scan 在序列长度 > 2K 时超过 FlashAttention-2,
- 比标准 scan 实现（ PyTorch 写循环） 20–40 倍

右图（ ）：

- Mamba 作为 RNN, $O(1)$ 计算（ KV 缓存）
- Transformer 需要 KV 缓存, GPU 内存,
- Mamba-6.9B 的吞吐量甚至超过了 5 倍小的 Transformer-1.3B!

5.6 模型消融 (Model Ablations)

原文翻译

我们对模型组件进行了一系列详细消融, $\approx 350\text{M}$ 模型在 Chinchilla token 数量下的语言建模设置

5.6.1 架构消融

Table 5: （消融： SSM 层。）Mamba 块与 H3 表现相似但更简单 在内部层中, LTI 模型参数化之间差异很小, SSM (S6) 提供了大幅改善

Model	Arch.	SSM Layer	Perplexity	Model	Arch.	SSM Layer	Perplexity
Hyena	H3	Hyena	10.24	-	Mamba	Hyena	10.75
H3	H3	S4 (complex)	10.30	-	Mamba	S4 (complex)	10.54
-	H3	S4 (real)	10.34	-	Mamba	S4 (real)	10.56
-	H3	S6	8.95	Mamba	Mamba	S6	8.69

通俗解释

表 5 的关键发现：

- 所有 LTI SSM (Hyena、S4-complex、S4-real) $\sim 10.2\text{--}10.8$,
- 换成 S6（ ） $\sim 8.7\text{--}9.0$, 1.5 个点——这是一个巨大的改善
- Mamba 架构比 H3 稍好 (8.69 vs 8.95) ,

Table 6: （消融： 。） Δ 是最重要的参数（ 1）,

Sel. Δ	Sel. B	Sel. C	Perplexity
$\times$	$\times$	$\times$	10.93
$\times$	$\checkmark$	$\times$	10.15
$\times$	$\times$	$\checkmark$	9.98
$\checkmark$	$\times$	$\times$	9.81
$\checkmark$	$\checkmark$	$\checkmark$	8.71

Table 7: （消融： A 的参数化。） SSM 中, S4D-Real 或随机初始化比标准 S4D-Lin 表现更好

A_n Initialization	Field	Perplexity
$A_n = -\frac{1}{2} + ni$	Complex	9.16
$A_n = -1/2$	Real	8.85
$A_n = -(n+1)$	Real	8.71
$A_n \sim \exp(\mathcal{N}(0, 1))$	Real	8.71

Table 8: （消融： Δ 投影维度。） ; 状态维度固定 $N = 16$ 。

Δ proj. size	Params (M)	Perplexity
-	358.9	9.12
1	359.1	8.97
2	359.3	8.97
4	359.7	8.91
8	360.5	8.83
16	362.1	8.84
32	365.2	8.80
64	371.5	8.71

Table 9: （消融： SSM 状态维度。）（上） B, C ; （下） B, C 。增加 SSM 状态维度 N 只有在 B, C 也是选择性时才显著改善性能 Δ 投影维度固定 64。

State dim. N	Params (M)	Perplexity
1	367.1	9.88
4	368.0	9.82
16	371.5	9.81
1	367.1	9.73
4	368.0	9.09
16	371.5	8.71

5.6.2 选择性参数消融

通俗解释

消融实验的核心结论:

表 6: Δ 的选择性最重要 ($10.93 \rightarrow 9.81$), (8.71)

表 7: ;

表 8: Δ 投影维度从 0 增到 64 带来从 9.12 到 8.71 的渐进改善

表 9 (!): B, C 固定时, N 几乎无用 ($9.88 \rightarrow 9.81$); B, C 选择性时, N 从 1 增到 16 带来超过 1.0 的困惑度改善 ($9.73 \rightarrow 8.71$), 1% 参数 这完美验证了”选择性 + 状态扩展”的核心动机

6 讨论 (Discussion)

原文翻译

相关工作 附录 C 讨论了选择机制与类似概念的关系 附录 D 有 SSM 和其他相关模型的扩展相关工作

没有免费午餐：-离散谱 结构化 SSM 最初被定义为连续系统的离散化，（视频）如第 4.1 和 4.5 节所述，（DNA）； LTI SSM 擅长的数据上的表现 我们关于音频波形的消融更详细地考察了这种权衡

下游能力 基于 Transformer 的基础模型（LLM），适配提示 上下文学习 指令调优 RLHF、量化等 我们特别感兴趣的是 SSM 等 Transformer 替代方案是否具有类似的属性和能力

缩放 我们的实验评估局限于小模型规模，LLM 的阈值（Llama 的 7B+），RWKV 和 RetNet 等在 7B 及以上规模评估的循环模型 Mamba 在更大规模下是否仍然表现优越有待评估

注意事项

论文的局限性（）：

- 1. 连续-离散权衡：，（LTI 的平滑归纳偏置在音频上更好）
- 2. 下游生态未验证：RLHF 等还未知
- 3. 规模限制：~3B 参数，7B+ 规模的表现未知

7 结论 (Conclusion)

原文翻译

我们引入了一种选择机制到结构化状态空间模型中，当融入一个简单的无注意力架构时，Mamba 在多种领域达到了最先进的结果，Transformer 模型的性能 我们对选择性状态空间模型在不同领域构建基础模型的广泛应用感到兴奋，音频和视频中 我们的结果表明 Mamba 是通用序列模型骨架的强候选者

致谢 (Acknowledgments)

原文翻译

我们感谢 Karan Goel、Arjun Desai 和 Kush Bhatia 对草稿的有益反馈

A 选择性 SSM 的机制 (Mechanics of Selective SSMs)

原文翻译

定理 1 的证明见第 4.5 节的数学推导框

B 选择性 SSM 的硬件感知算法

原文翻译

没有输入依赖的选择性, SSM 可以高效实现为卷积, (FFT) 有了选择性, SSM 不再等价于卷积, 我们描述如何使用核融合和重计算使 SSM scan 既快速又内存高效

速度 标准实现方式是在 GPU HBM 中准备大小 (B, L, D, N) 的 scan 输入 $\overline{A}, \overline{B}$, (B, L, D, N) 的输出写回 HBM, C 产生 (B, L, D) 的输出 这需要 $O(BLDN)$ 量级的内存读写
我们的融合方案:

1. 从慢速 HBM 读入 $O(BLD + DN)$ 字节的数据 (Δ, A, B, C) SRAM。
2. 在 SRAM 中离散化, (B, L, D, N) 大小的 $\overline{A}, \overline{B}$ 。
3. 在 SRAM 中执行并行结合扫描,
4. 与 C 相乘并求和, (B, L, D) 的输出并写回 HBM。

这样 IO 量减少了 $O(N)$ 倍, 20–40 倍

内存 前向传播中不保存中间状态 ((B, L, D, N)) 反向传播时重新计算这些状态——因为输入和输出梯度的大小都是 $O(BLN + DN)$, HBM 读取 $O(BLND)$ 元素的开销 结果: SSM 层的内存需求与使用 FlashAttention 的优化 Transformer 相当 每个注意力层存约 12 字节/token 激活, MLP 层约 20 字节/token, 32 字节 每个选择性 SSM 存约 16 字节/token, 32 字节——完全匹配

C 讨论:

原文翻译

我们的选择机制受到门控 超网络和数据依赖等概念的启发并与之相关

门控 (Gating) 最初指 LSTM 和 GRU 等 RNN 的门控机制, 但“门控”一词已被泛化为任何乘法交互 () 我们认为原始的 RNN 门控与流行的乘法门控有非常不同的语义含义

超网络 (Hypernetworks) 指参数本身由更小网络生成的网络

数据依赖 指模型某些参数依赖于数据

GLU 激活的例子 考虑对角线性层 $y = Dx$, $D = \sigma(Wx)$ 。这产生 $y = \sigma(Wx) \circ x$ ——即 GLU 函数 这技术上满足门控 超网络和数据依赖的定义,

选择 (Selection) 我们将其视为与传统 RNN 门控最密切相关, (1) 我们使用”选择”而非”门控”来避免后者的过度使用 更狭义地, 机制性动作

D 扩展的相关工作 (Extended Related Work)

原文翻译

S4 变体包括: S4 (SSM) DSS (SSM 的有效性) S5 (S4, MIMO 降低了状态维度; S6 保持 SISO 以获得更大有效状态) Mega (EMA 简化) Liquid S4 () SGConv/Hyena/LongConv 等 (,)
SSM 架构包括: GSS (SSM 架构,) H3 (S4 + 线性注意力) RetNet (+ 简化 SSM, $N = 1$ 但大头维度) RWKV (AFT 的 RNN, WKV 机制是两个 SSM 的比值)
与 RNN 的关系: QRNN、SRU 等是无时间非线性的门控 RNN, SSM 的特例, ($N = 1$) B, C ,
线性注意力: RFA、Performer、TransNormer、cosFormer 等核注意力变体
长上下文模型: Recurrent Memory Transformer、LongNet、HyenaDNA、Sparse Transformer 等,

E 实验细节 (Experimental Details)

E.1 合成任务细节

原文翻译

选择性复制 序列长度 4096, 16, 16 个数据 token。2 层模型, $D = 64$ 。训练 400K 步, 0.0001, 64。
归纳头 训练序列长度 256, 16。2 层模型 Mamba 维度 $D = 64$, $D = 128$ 。

Table 10: (归纳头。) $2^8 = 256$ 上训练, ✓ 表示完美泛化, ✗ 表示内存不足

Model	Params	2^6	2^8	2^{10}	2^{12}	2^{14}	2^{16}	2^{18}	2^{20}
MHA-Abs	137K	✓	100.0	26.6	9.8	7.8	✗	✗	✗
MHA-RoPE	137K	✓	100.0	31.3	8.6	5.5	✗	✗	✗
MHA-xPos	137K	✓	100.0	67.6	7.0	7.8	✗	✗	✗
H3	153K	✓	100.0	39.5	14.8	5.9	8.2	8.2	7.4
Hyena	69M	✓	100.0	44.1	6.6	7.0	6.6	5.9	9.8
Mamba	74K	✓	100.0	✓	✓	✓	✓	✓	✓

E.2 语言建模细节

原文翻译

所有模型使用 AdamW 优化器: 1.0, 0.1, dropout, 改进配方 (Transformer++、H3++、RetNet、Mamba) : GPT3 值的 5×; ; RMSNorm 替代 LayerNorm; AdamW $\beta = (0.9, 0.95)$ 。

Table 11: (缩放定律模型大小。)

Params	n_layers	d_model	n_heads/d_head	Steps	LR	Batch	Tokens
125M	12	768	12/64	4800	6e-4	0.5M	2.5B
350M	24	1024	16/64	13500	3e-4	0.5M	7B
760M	24	1536	16/96	29000	2.5e-4	0.5M	15B
1.3B	24	2048	32/64	50000	2e-4	0.5M	26B

E.3 附加缩放定律消融

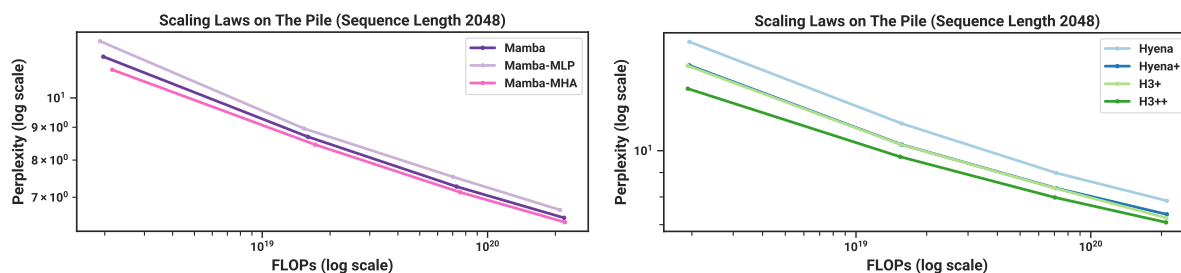


Figure 10: (缩放定律：)。 (左) Mamba 架构的交错变体 (右) H3 架构的训练配方消融

原文翻译

Mamba 架构： 我们将 Mamba 块视为标准 SwiGLU 块加上额外的 $\text{conv} \rightarrow \text{SSM}$ 路径 两种消融：(1) Mamba 块与标准 MLP 块交错 (= 去掉一半 SSM)；(2) Mamba 块与 MHA 块交错 (= Transformer++ 的 MLP 中加 SSM)

结果： Mamba-MLP 只略差， Transformer++ 外的所有模型 Mamba-MHA 只略好——这令人惊讶， LTI SSM + 注意力的组合能带来实质改善

H3 架构： 改进训练配方带来大幅改善 内部 LTI SSM 的选择 (Hyena vs S4)， 头维度扩展改善性能，

E.4 DNA 建模细节

Table 12: (类人猿 DNA 分类。) 2^{10} 到 2^{20} 长度序列上微调后准确率 随机猜测为 20%。

Model	Params	2^{10}	2^{12}	2^{14}	2^{16}	2^{18}	2^{20}
HyenaDNA	1.4M	28.04	28.43	41.17	42.22	31.10	54.87
Mamba	1.4M	31.47	27.50	27.66	40.72	42.41	71.67
Mamba	7M	30.00	29.01	31.48	43.73	56.60	81.31

E.5 音频细节

原文翻译

图 11 显示， S4 到 S6 的选择机制变化在长音频波形上并非总是有益的。在长音频上它实际上显著损害了性能——这从音频是均匀采样且非常平滑 因此受益于连续 LTI 方法的角度来看是直觉的

然而， ， ： LTI 的， ”分词化”和压缩后， LTI 的

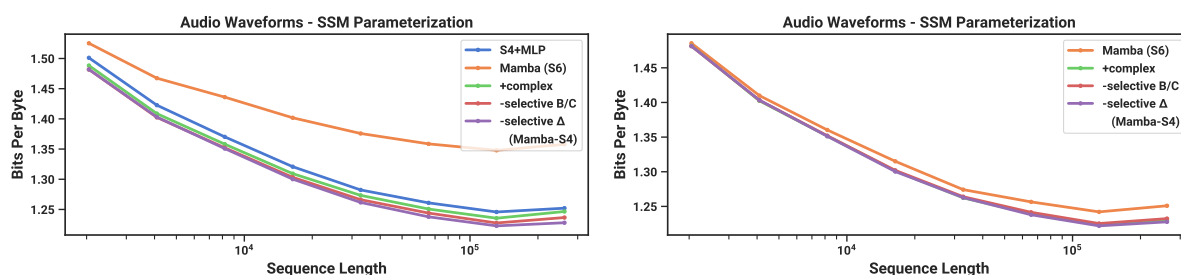


Figure 11: (音频预训练消融。) “连续”信号模式, LTI 模型 (左) (右)
U-Net 块

通俗解释

这是论文中“没有免费午餐”的具体体现：

- 音频是“连续”数据：平滑 有强时间规律性 LTI ()
- 选择性在外层有害：S6 的选择机制会打破时间平滑性，
- 但在内层无害：U-Net 的池化压缩后， “离散 token”，
- 最佳策略：LTI (S4), (S6)——结合两者优势

E.6 效率基准细节

Table 13: (内存基准。) Mamba 的内存占用与最优化 Transformer 相当 125M 模型的结果

Batch size	Transformer (w/ FlashAttention-2)	Mamba
1	4.6GB	4.8GB
2	5.2GB	5.8GB
4	6.9GB	7.3GB
8	11.5GB	12.3GB
16	20.7GB	23.1GB
32	34.5GB	38.2GB

原文翻译

表 13 显示 Mamba 的内存需求与使用极优化实现 (torch.compile + FlashAttention-2) Transformer 相当 具体而言, 12 字节/token, MLP 层约 20 字节/token, 32 字节; SSM 约 16 字节/token, 32 字节——完全匹配

Table 14: YouTubeMix 长度缩放的序列长度和批大小

Sequence length	Batch size	Tokens/batch
958464	1	958464
479232	2	958464
239616	4	958464
120832	8	966656
61440	16	983040
30720	32	983040
16384	64	1048576
8192	128	1048576

F 参考文献一览表

以下表格列出论文中主要引用的文献及其在本文中的角色

文献	主要内容	在本文中的引用原因
Vaswani et al. 2017 (Transformer)	提出 Transformer 架构和自注意力机制	主要对比基线架构
Gu et al. 2022 (S4)	提出第一个高效的结构化状态空间模型	Mamba 的直接前身,
Gu et al. 2020 (HiPPO)	提出 HiPPO 理论用于在线记忆	SSM 初始化的理论基础
Dao et al. 2023 (H3)	提出 H3 架构 (SSM + 线性注意力)	Mamba 架构的直接基础
Poli et al. 2023 (Hyena)	用 MLP 参数化全局卷积替代 S4	重要基线模型
Dao et al. 2022 (FlashAttention)	GPU 内存感知的高效注意力	硬件感知算法的灵感来源
Smith et al. 2023 (S5)	第一个用并行扫描计算的 S4	并行扫描的先驱, S6 与之对比
Gu & Dao 2022 (S4D)	S4 的对角参数化	初始化方案和对角结构
Touvron et al. 2023 (LLaMa)	高效开源 LLM	Transformer++ 配方的来源
Brown et al. 2020 (GPT-3)	大规模语言模型缩放	模型尺寸和训练配方的参考
Biderman et al. 2023 (Pythia)	开源 LLM 套件	主要语言建模基线
Peng et al. 2023 (RWKV)	基于线性注意力的 RNN	重要基线 (SSM 类 RNN)
Sun et al. 2023 (RetNet)	带保留机制的序列模型	基线模型
Nguyen et al. 2023 (HyenaDNA)	DNA 基因组学基础模型	DNA 实验的主要基线
Goel et al. 2022 (SaShiMi)	S4 驱动的音频生成	音频实验的主要基线

文献	主要内容	在本文中的引用原因
Blelloch 1990 (Parallel Scan)	并行前缀和算法	选择性 SSM 高效计算的核心
Olsson et al. 2022 (Induction Heads)	LLM 上下文学习的机制解释	合成任务的来源
Hoffmann et al. 2022 (Chinchilla)	训练 token 的最优缩放定律	缩放实验协议
Katharopoulos et al. 2020	线性注意力框架	SSM 与注意力的联系
Ma et al. 2023 (Mega)	指数移动平均 + 注意力混合	实值 SSM 的先驱
Bradbury et al. 2016 (QRNN)	准循环神经网络	与选择性 SSM 最密切相关的经典 RNN